

EDUHACKERS

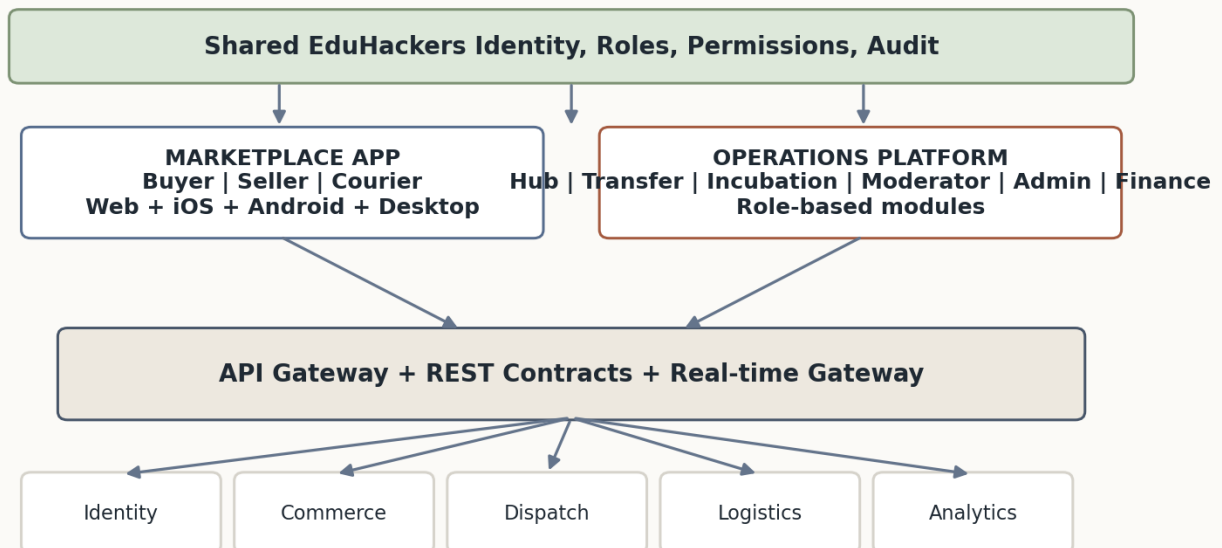
Developer Build, Workflow and Technical Research Dossier

System design research for the Eduvos Marketplace concept: Uber-style dispatch, campus hubs, seller/courier workflows, technology stack, database model, APIs, GitHub workflow, POF build plan and cross-functional UX questions.

Status: Research hypothesis pack - not a final implementation specification. Built for Paballo Kali, Developer / Research Lead, to brief the EduHackers team.

EduHackers platform split: Marketplace + Operations

Research framing: two application families, one backend platform.



Prepared for: EduHackers team review. Primary alignment: Eduvos Hack Jam marketplace challenge and all-campus operating model.

1. Executive summary

This dossier continues the earlier EduHackers marketplace research, but shifts the lens to the developer, build workflow and technical architecture. The question is not only what the product should do. It is how the system should work, how different parts should communicate, what should be built first, what should be researched further, and how the team should manage engineering work from GitHub to prototype testing.

The central technical hypothesis is that EduHackers should adapt on-demand marketplace patterns, especially dispatch, live status and dynamic fulfilment, without simply copying Uber, Uber Eats or Mr D. Those platforms already prove the value of real-time matching, courier tracking, busy modes, live order states and dynamic supply-demand balancing. EduHackers differs because it must combine buyer demand, student sellers, campus hubs, QR custody, hub-to-hub transfers, student couriers, staff operations, marketplace payment logic and Incubation Hub intelligence inside a closed Eduvos environment.

The recommended research architecture is one EduHackers platform with two application families: a student-facing Marketplace and a role-based Operations Platform. Both should share one identity system, one backend domain model, one API contract set and one event model, while still providing very different user experiences for buyers, sellers, couriers, hub operators, transport staff, moderators, admins, finance operations and Incubation Hub staff.

The hardest proof is not visual polish. The hardest proof is state correctness: only one seller can win a request, only one courier can take a delivery, a service slot cannot be double-booked, a package must have a reliable custody chain, a payment/ledger split must be auditable, and every real-time screen must recover correctly when connectivity drops. The Proof of Flow should demonstrate those system behaviours before the team attempts the full super-app.

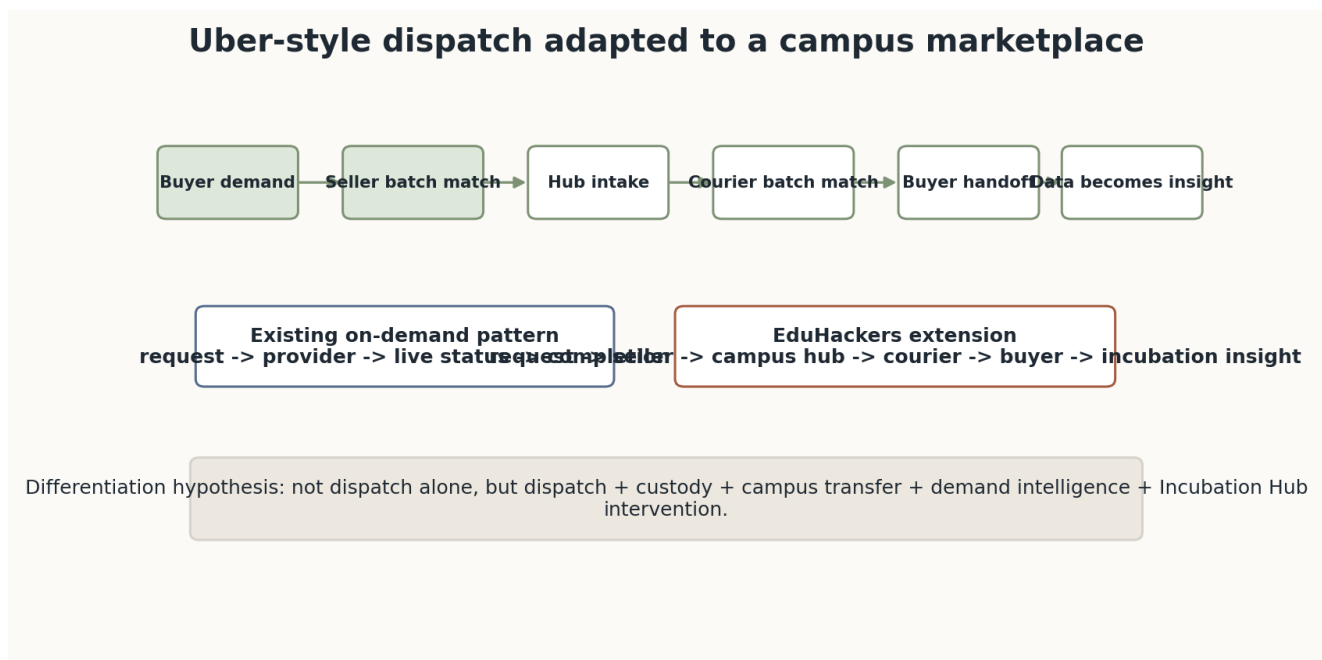


Figure 1. EduHackers should borrow proven on-demand marketplace engineering patterns, then adapt them to campus-specific logistics and entrepreneurship intelligence.

2. Research scope and decision discipline

This document is research. It records the current best hypotheses based on team discussion, existing platform patterns and available technical documentation. It is not yet a final build specification. The purpose is to give the developer, product lead, UX/UI designer, business analyst and pitch lead a shared map of what needs to be tested, what should be built in a POF, and what requires Eduvos, payment-provider or legal confirmation.

The research method used in this dossier is comparative and system-oriented. For each major feature, the report asks what already exists in similar platforms, what EduHackers can learn from it, where the existing pattern does not fit Eduvos, what EduHackers should research as its own hypothesis, and what evidence would prove or disprove that approach.

The repeated rule is: do not turn a brainstormed idea into a final requirement until it survives a technical feasibility check and a user/operations validation check. Where the team has selected a direction, this report marks it as a working hypothesis unless the decision depends purely on internal project preference.

Evidence labels

Label	Meaning	How it should be treated
Confirmed external pattern	A similar pattern is visible in existing market or official documentation	Can be used as prior art, not as proof that EduHackers users want it
Team-selected hypothesis	A direction chosen during developer research discussion	Build/test as a design hypothesis
Needs Eduvos confirmation	Requires institutional access, policy, operations or IT confirmation	Do not promise as final
Provider-dependent	Depends on payment, auth, SMS, push, map or storage provider support	Keep behind adapter interfaces
POF candidate	Should be proven in the proof-of-flow build	Prioritise in technical prototype

3. Product system overview

The strongest architecture is not one application with everything hidden in menu tabs. EduHackers should be researched as a platform with two application families. The Marketplace is used by students as buyers, sellers and couriers. The Operations Platform is used by Hub Operators, Transport Operators, Incubation Hub staff, moderators, finance operations and admins. These apps share the same backend services but expose different workflows, data, security controls and dashboards.

This separation matters because the Marketplace must feel quick and student-friendly, while Operations must be reliable, auditable and role-specific. A Hub Operator needs scan queues and custody states, not a shopping feed. Incubation staff need demand intelligence and business health indicators, not courier live-trip controls. Admins need policy configuration, not buyer checkout screens.

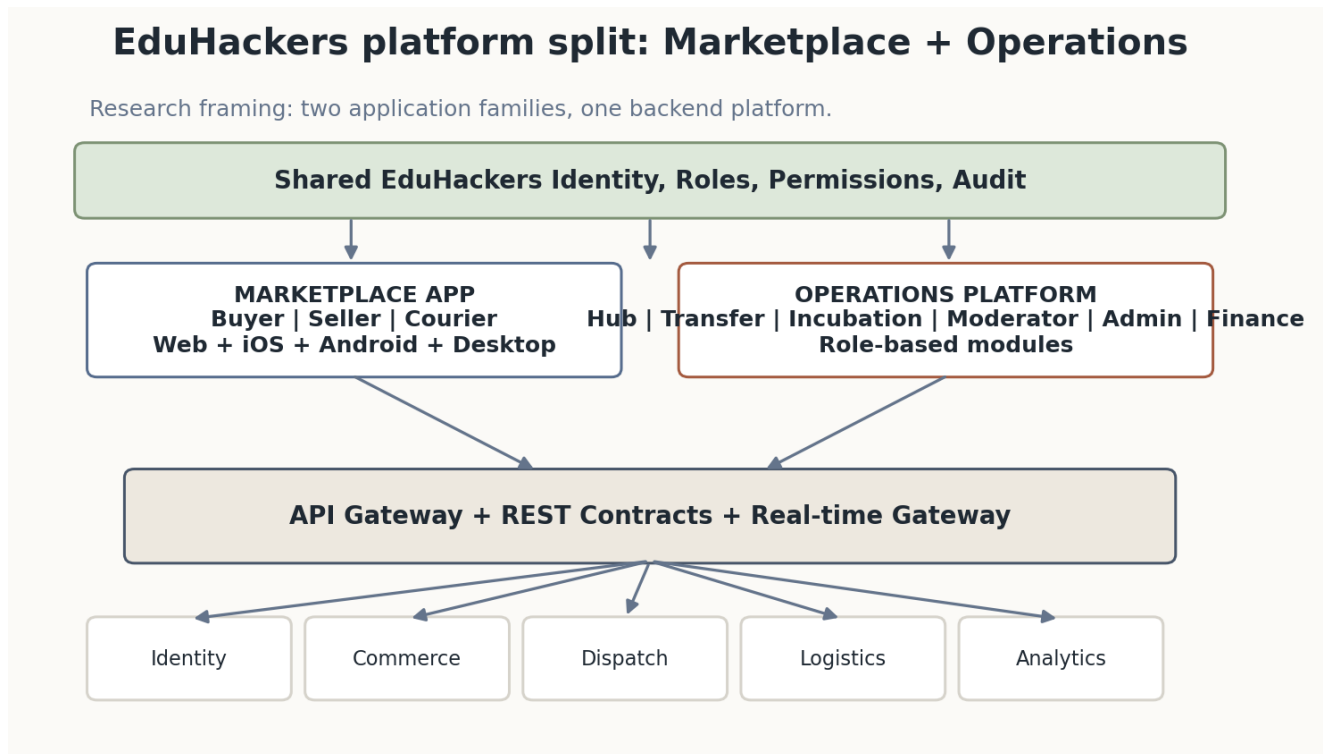


Figure 2. One identity and backend platform supports two application families. This prevents separate, inconsistent systems while keeping UX and permissions clear.

Application family responsibilities

Application family	Primary roles	Main purpose	Key developer concern
Marketplace	Buyer, Seller, Courier	Student commerce, requests, bookings, delivery work, messaging, earnings	Fast live flows, capacity, payments, identity, app permissions
Operations Platform	Hub Operator, Transport Operator, Incubation, Moderator, Finance, Admin	Package custody, transfer manifests, disputes, interventions, policy configuration	RBAC, audit, staff security, operational dashboards
Shared backend	All roles through API	Single source of truth for order state, payment ledger, package custody and identity	No trust in client-side role claims; strong transactions and event consistency

4. Uber-style dispatch model, adapted

The team asked whether EduHackers can work like Uber: a client requests something, matching participants receive live offers, the first suitable participant accepts, and the system locks the job. The research answer is yes, but the match target is not only a driver. EduHackers must match buyers to sellers, sellers to hubs, hubs to couriers, transfer batches to transport capacity and service buyers to available seller slots.

The platform should not be a bidding marketplace. Sellers list fixed prices. Buyers request a listed product or service. The backend uses the listing price snapshot and availability at the time of request. No counter-offers or buyer proposed budgets are involved in the Uber-style flow.

Seller matching hypothesis

When a buyer requests a listed item or service, the system ranks eligible sellers by campus, category, stock or availability, trust standing, capacity, response reliability and fulfilment suitability. Instead of notifying one seller at a time, the current research direction is small staggered batches. Batch 1 may notify a few best-fit sellers over 30 to 60 second intervals. If no one accepts, the next batch expands. If local supply fails, the system expands to the next viable campus.

Once any seller accepts, the backend immediately locks the order to that seller and cancels all other open offers. This acceptance must be atomic. If several sellers tap accept at the same time, one backend transaction must win and every other response must receive a "request already accepted" result.

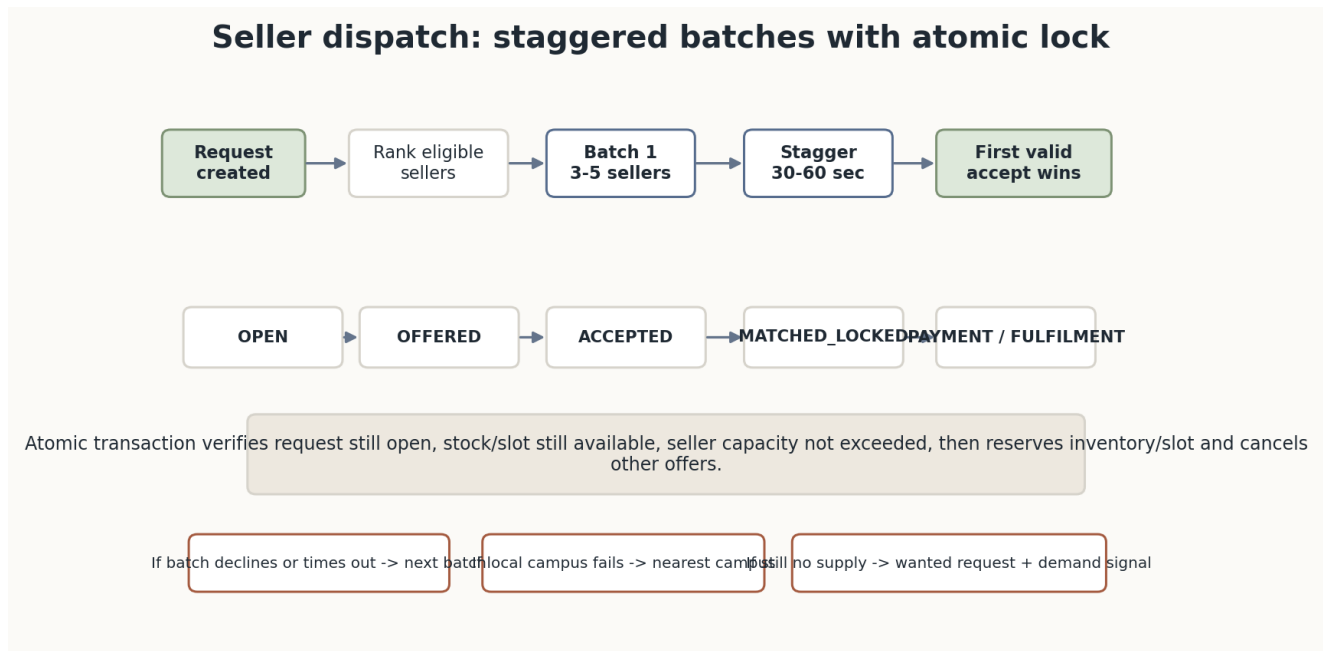


Figure 3. Seller dispatch relies on ranked batches, staggered notifications and atomic locking. The frontend cannot be trusted to prevent race conditions.

5. Seller dispatch state model

The state model needs to be explicit because dispatch bugs become trust bugs. The buyer should not see multiple sellers accepting the same request. Sellers should not prepare orders that were already won by someone else. Stock should not be oversold. Service slots should not be double-booked. The backend must own these transitions.

Working seller dispatch states

State	Meaning	Key system action
REQUEST_CREATED	Buyer has submitted a product/service request	Create request record, snapshot listing price, classify category
SELLER_MATCHING	Matching engine is ranking eligible sellers	Filter by campus, category, stock/slot, capacity, schedule, good standing
OFFERED_TO_BATCH	Batch of sellers is receiving staggered notifications	Create SellerOffer records with expiry times
SELLER_ACCEPTED	A seller has responded positively	Begin atomic lock transaction
MATCHED_LOCKED	Request is assigned to one seller	Cancel other offers, reserve stock/slot, begin payment/fulfilment path
PAYMENT_PENDING	Product buyer must pay after seller accepts	Start payment countdown and release on expiry
FULFILMENT_PENDING	Payment is valid and seller must fulfil	Apply category-specific fulfilment strategy
EXPIRED_OR_DECLINED	Offer batch failed	Move to next batch, next campus or request/opportunity signal

The seller matching model should always prefer continuation over dead ends. If the local campus cannot fulfil the request, the engine expands to the next campus, an alternative seller, a substitute time slot, a hub-to-hub option, or a wanted-request flow that becomes demand intelligence.

6. Seller capacity and Busy Mode research

Existing merchant platforms show that high order load needs operational controls. DoorDash allows merchants to slow or pause orders through its status controls, and Uber Eats provides Busy Mode, preparation-time tools and pause capabilities for stores that are overloaded. These patterns support the EduHackers hypothesis that student sellers need workload controls rather than being expected to accept every request indefinitely. [R5] [R6]

The EduHackers version is adapted to student sellers. Instead of only adding prep time, the platform should enforce a maximum number of simultaneous accepted orders per category and an overall account cap. The team selected 15 maximum active orders per category and 25 maximum active orders overall as research hypotheses. These caps protect students from overload and protect buyers from delays.

Seller availability and workload rules

Rule	Research direction
Per-category capacity	Each category can have its own active-order limit, never above 15
Overall capacity	Seller account maximum of 25 active accepted orders across all categories
Busy Mode	Seller can reduce effective capacity manually using Normal, Busy, Very Busy, Pause or custom limit
At Capacity	Listings remain visible but seller is excluded from new matching
Notify Me queue	Buyers can subscribe, but alternatives are still shown; queue notifications are staggered
Scheduling	Sellers set recurring availability per category; no one-off exceptions for the research baseline
Reliability recovery	Bad standing recovers through consecutive successful orders and negative-event decay over time

The matching engine should calculate effective capacity as the minimum of platform policy, temporary restriction and seller-selected capacity. A seller under a temporary category reduction can still reduce capacity further using Busy Mode, but cannot override the system maximum upward.

7. Product fulfilment: campus hub model

For normal physical products, the preferred research model is not seller-to-courier direct handoff. The seller drops the packaged item at a designated campus hub. The hub scans the package, records condition and accepts custody. Only after the package is physically confirmed at the hub does courier dispatch begin. This prevents couriers from travelling to collect products that are not actually ready.

This hub-and-spoke model is a major EduHackers differentiation. Standard delivery apps often connect courier directly to merchant. EduHackers researches a campus custody layer, which can improve scheduling, traceability, privacy and safety in a closed university environment.

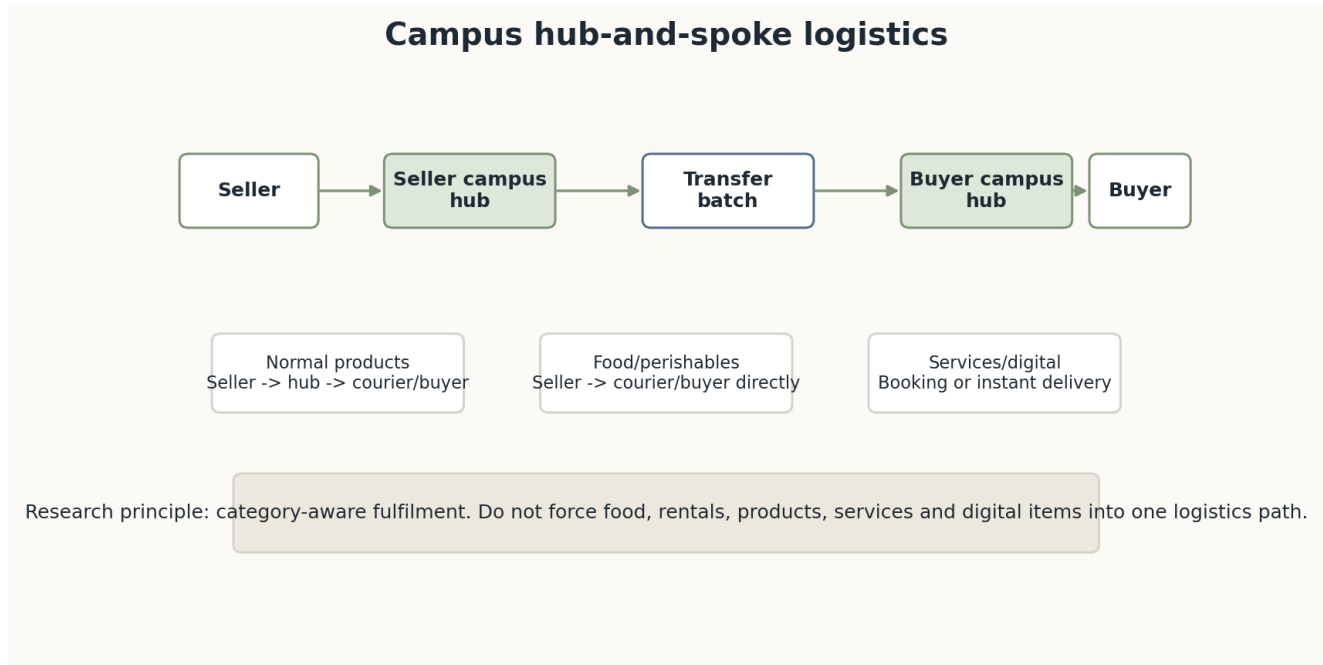


Figure 4. Fulfilment should be category-aware. Normal products use hubs; food can use direct pickup; services and digital products use other flows.

Normal product flow

Step	System state	Notes
Seller accepts	SELLER_ACCEPTED	Order locks to seller; price snapshot preserved
Buyer pays	PAYMENT_CONFIRMED	If buyer fails to pay, request returns to matching pool
Seller prepares	SELLER_PREPARING	1-2 hour default research window, dynamic by category
Seller drops at hub	HUB_INTAKE_STARTED	Hub operator scans QR and inspects package
Hub accepts	HUB_RECEIVED	Courier dispatch becomes eligible only now
Courier collects	COURIER_COLLECTED	Courier scans QR; custody moves from hub to courier
Buyer receives	PIN_VERIFIED	Buyer one-time PIN confirms recipient
Completed	PAYOUT_ELIGIBLE	Ledger releases seller/courier payments according to rules

8. Package QR, PIN and custody model

The chain of custody should use two different proof types. The package QR proves that the physical package is the correct package. The buyer PIN proves that the authorised buyer or recipient received it. The QR should contain only an opaque token. It must not expose buyer address, payment details, phone number or identity data.

At every custody transfer, the receiving party records package condition. The official condition baseline starts when the Hub Operator receives the package, not with seller photos. If a package is poorly packaged, unsealed, leaking, visibly damaged or unsuitable for transport, the hub can reject it before custody is accepted.

Chain of custody: QR identifies the package; PIN verifies buyer handoff

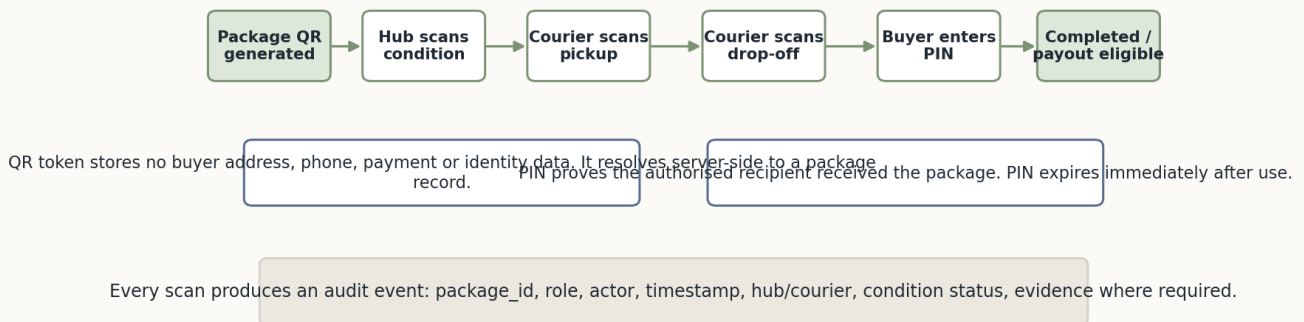


Figure 5. QR and PIN serve different proof functions. QR confirms the object; PIN confirms the recipient.

Package rejection and correction

If hub intake rejects a package, the seller receives a 24-hour grace period to fix and return it. The buyer, seller and relevant logistics participants are notified of what is happening. If the seller does not return an acceptable package within 24 hours, the order is cancelled, buyer payment is refunded/released and the missed correction deadline is recorded against seller reliability.

Repeated packaging issues should trigger Eduvos-approved learning resources, but training completion must remain educational only. It should not give reliability points or automatically restore standing. Marketplace reliability should be driven by marketplace behaviour, not whether a resource was completed.

9. Food and perishable fulfilment

Food should not be forced through the hub flow. The team correctly identified that prepared meals and perishable goods may degrade or create safety issues if held at a pickup hub. For food, the courier workflow should begin immediately after seller acceptance and payment confirmation, while the seller is still preparing the food. The courier travels toward the seller so pickup can happen as soon as the food is ready.

Uber Eats documentation emphasises preparation-time accuracy, Busy Mode and ready-for-pickup behaviour to reduce courier wait time. EduHackers can learn from this while adapting the model to student sellers, campus geography and category-specific food compliance. [R6] [R7]

Food timeline research

Seller timeline	Courier timeline	Buyer view
ACCEPTED	COURIER_MATCHING	Order accepted; estimated preparation time
PREPARING	COURIER_ASSIGNED / TO_SELLER	Courier approaching seller
READY_FOR_PICKUP	COURIER_AT_SELLER	Preparing / ready status updates
HANDED_TO_COURIER	IN_TRANSIT	Live location and ETA
COMPLETED	PIN_VERIFIED	Delivery confirmed and receipt available

If the courier reaches the seller and the food is still not ready, the courier waits 10 to 15 minutes. After that threshold, the courier can choose to keep waiting with possible waiting compensation, or release the job without reliability penalty. If released, the system immediately rematches another courier using the updated ready-time estimate.

10. Courier platform model

All courier workflows should share one courier platform capability set. Live tracking, ETA, in-app messaging, masked or in-app calling, one-time PIN confirmation, earnings records, reliability events and payout logic should be core courier infrastructure. Food, normal products and transfer routes differ in pickup strategy, not in basic courier capability.

Transport modes

Mode	Eligibility research	Likely delivery scope
Walking	Eduvos identity + courier onboarding	Same-campus or very-nearby delivery only
Bicycle	Basic courier onboarding; campus policy check	Same-campus and nearby local radius
Motorbike	Relevant transport documentation where required	Wider local radius; optional transfer eligibility
Car	Licence/vehicle checks where required	Wider local radius; eligible for inter-campus transfer mode

The team selected one active transport method per courier account at a time. If a courier changes transport mode, new checks may be required. Walking and bicycle should be limited to campus/nearby jobs, while motorbike and car can qualify for broader routes and transfer mode where approved.

Courier reliability

If a courier accepts a job and then does not move toward the hub for a configurable period, the system should warn them first and then release the job back into matching if there is still no progress. Repeated accept-then-cancel, missed pickup deadlines or unexplained inactivity should reduce future matching priority and can eventually cause a temporary delivery pause. The architecture must distinguish GPS/connectivity problems, campus access issues, buyer delays and courier behaviour.

11. Same-campus delivery and failed handoff

For normal product delivery, the courier receives the package from the campus hub, not directly from the seller. If the courier reaches the buyer and the buyer is unavailable, the courier waits 10 to 15 minutes while the buyer and courier can communicate via app chat/call. After that, the package returns to the designated campus hub rather than to the seller directly.

Failed delivery state model

State	Meaning
AT_DROPOFF	Courier has reached buyer location or chosen handoff point
WAITING_FOR_BUYER	10-15 minute buyer-unavailable timer is running
BUYER_UNAVAILABLE	Buyer did not complete handoff in time
RETURNING_TO_HUB	Courier is taking package back to campus hub
HUB_RETURN_RECEIVED	Hub scans package back into custody
AWAITING_BUYER_COLLECTION	Buyer has 48 hours to collect from hub
UNCOLLECTED	48-hour window expired
SELLER_RECOLLECTION	Seller is notified to collect package from hub

The default hub holding period is 48 hours, with category/value exceptions. After the window expires, the package returns to seller collection and failed-delivery/handling fees can apply where appropriate. Food and perishables need separate rules because hub storage may be unsuitable.

12. Hub-to-hub transfer model

For cross-campus fulfilment, the research direction is hub-to-hub transfer rather than direct long-distance courier delivery to each buyer. The package moves from seller campus hub to destination campus hub through scheduled transfer windows, then the buyer can collect or use local last-mile delivery.

The team selected three scheduled transfer windows per day as a research hypothesis. Eduvos-approved transport should be prioritised as the scheduled backbone, with verified student transfer couriers available for overflow. If Eduvos-approved capacity is full, the buyer can either wait for the next standard window or request an earlier student-courier overflow transfer and pay the extra charge only if a courier accepts.

Transfer lifecycle

Stage	State	Notes
Origin intake	SELLER_HUB_RECEIVED	Origin hub accepts package and records condition
Batch assignment	WAITING_FOR_TRANSFER_WINDOW	Package assigned to morning/afternoon/evening window
Manifest lock	TRANSFER_MANIFEST_LOCKED	No more packages after departure confirmation
Origin scan	TRANSFER_CUSTODY_ACCEPTED	Transfer courier scans every package before departure
Travel	IN_TRANSFER	Route is hub-to-hub; no buyer private details exposed
Destination reconcile	MANIFEST_RECONCILED	Destination hub scans every package
Final mile	READY_FOR_PICKUP or LOCAL_COURIER	Buyer gets hub pickup or local delivery options

Transfer couriers see package count, package ID, QR, package/handling type and destination hub. They should not see buyer/seller private details, payment value or home address. Every package must be scanned at origin and destination. Transfer payout becomes eligible only after destination reconciliation.

13. Transfer exceptions and custody evidence

If a destination Hub Operator finds that a package is missing, damaged, tampered with or does not match the manifest, the whole transfer payout should initially be held. After the affected packages are identified, unaffected packages can be completed and the related portion of courier earnings released. Only the disputed portion remains held while the investigation continues.

The system should not assume courier fault. The purpose of QR scans, custody events and condition checks is to reconstruct where an issue likely occurred. Every physical handoff should produce a condition record: sealed/intact, damaged, opened/tampered, packaging compromised or other exception. Exceptions should require photos and notes.

Exception data objects

Object	Purpose
PackageCustodyEvent	Records who transferred custody to whom, where and when
PackageConditionCheck	Records condition status, notes and evidence at custody event
TransferBatch	Groups packages for a scheduled route
TransferEarningAllocation	Allocates courier transfer earnings to the route and packages
PackageDispute	Investigates missing, damaged, tampered or mismatched packages
AuditLog	Preserves staff/courier actions and permission-sensitive changes

14. Services and appointment booking

Services should not behave like products. The research model uses broad service availability windows plus bookable slots inside those windows. Buyers do not manually browse a calendar by default. Instead, the system proposes the next available suitable slot, then asks the buyer to confirm it before booking is final.

The proposed service slot is temporarily held for 5 to 10 minutes, dynamically adjusted by service type, urgency and scarcity. While held, the slot is unavailable to other buyers. If the buyer rejects the proposed slot, the system proposes another slot rather than forcing them into a complex calendar.

Service booking flow

Step	State	Research behaviour
Request	SERVICE_REQUESTED	Buyer chooses service, campus/scope and category
Match	MATCH_SELLER_SLOT	Engine ranks seller-slot combinations
Propose	SLOT_TEMPORARILY_HELD	System presents next available slot
Confirm	BUYER_CONFIRMED	Buyer accepts proposed time
Authorise	PAYMENT_AUTHORISED	Service funds are protected before seller acceptance
Seller response	SELLER_ACCEPTED or SELLER_DECLINED	Seller confirms or declines
Decline	REMATCH_OFFERED	Buyer decides whether to rematch; system does not silently switch seller
Complete	BUYER_CONFIRMED_SERVICE	Payment released according to service rules

If a replacement seller is more expensive, the research preference is to authorise only the difference where the payment provider supports it. If the replacement is cheaper, the buyer can choose refund/release to original payment method or EduHackers credit, subject to provider and policy feasibility.

15. Payment, wallet and ledger research

Payment is one of the highest-risk technical areas. EduHackers should not literally hold money itself unless a regulated arrangement permits it. The safer research architecture is that EduHackers stores transaction states and ledger entries while a payment provider remains the system of record for authorisation, capture, refund, split settlement and payout where applicable.

Paystack supports transaction split APIs that let merchants split settlements across payout accounts and subaccounts, which makes it relevant prior art for marketplace research in South Africa. [R13] The architecture should remain provider-agnostic because Ozow, Payfast, Paystack, bank EFT and future wallet partners may differ in authorisation, refunds, incremental payments, splits and payout capability.

Ledger principles

Principle	Reason
Never store raw card/bank credentials	Use provider references and masked details only
Separate seller amount, courier amount, platform fee and processor fee	Avoids refund and reconciliation confusion
Snapshot price and fee rules at checkout	Enables audit when rules change later
Store every financial movement as a ledger entry	Allows dispute, payout and refund reconstruction
Keep wallet/credit provider-dependent	Withdrawable balances introduce regulatory/payment complexity
Minimum withdrawal R50	Team-selected threshold to reduce payout overhead on tiny balances

Credit and withdrawals

EduHackers credit can be used across the ecosystem: products, services, delivery fees, transfers, rentals, tickets and other approved charges. The team selected withdrawable credit with a R50 minimum withdrawal. This requires careful research because withdrawable stored value looks materially different from a simple non-cash coupon. The safer approach is to integrate a suitable payment/wallet partner rather than making EduHackers the regulated store-of-value operator.

Users may save up to three verified payout destinations. Adding or changing a payout destination requires step-up verification. Once verified, it can be used for instant withdrawal immediately, with suspicious withdrawals retained as an exception-path security review rather than part of normal UX.

16. Fees and dynamic pricing research

The team selected a variable fee model rather than one fixed fee for everyone. Fees can vary by category, order value, transaction type, Trusted Seller/Courier status, delivery complexity and subsidy rules. The platform can take fees from both seller transactions and courier delivery fees because it carries matching, payment, operations, infrastructure and support costs.

Existing on-demand platforms use dynamic supply-demand mechanisms, but EduHackers should avoid opaque surge-style pricing. Uber describes surge as a real-time response to demand/driver imbalance, while telling riders the higher price before commitment. [R4] EduHackers should research a more bounded and transparent approach: seller prices remain fixed by sellers, while delivery/transfer incentives can adjust within admin-defined caps based on distance, urgency, courier supply and campus route capacity.

Pricing engine layers

Layer	Controlled by	Examples
Seller listing price	Seller	Product/service base price; not changed by platform demand
Base delivery/transfer fee	Policy engine	Same-campus delivery, cross-campus transfer, pickup
Dynamic modifiers	Pricing engine within caps	Courier shortage, urgent delivery, distance/time, transfer overflow
Platform fees	Fee rules	Seller fee, courier-side fee, caps, discounts, Trusted Seller benefits
Subsidies	Admin/Eduvos policy	Incubation-supported seller, launch promotion, campus campaign
Payment processor fees	Provider	Card/EFT/pay-by-bank costs, payout fees

Every final quote should store a pricing snapshot with rule version, modifiers, final price, expiry and what was shown to the user. This prevents disputes where the current system recomputes an old price differently.

17. Identity, verification and roles

EduHackers should use one identity system for both Marketplace and Operations. A user starts as an Eduvos-verified buyer. Government ID verification is required only when activating Seller or Courier mode. If a student verified ID for Seller mode and later activates Courier mode, the ID verification is reused; only courier-specific checks are added.

For recurring login, the research should consider passkeys and device biometrics rather than building a custom facial-recognition database. FIDO passkeys use device biometric/PIN unlock while the server verifies cryptographic proof, which reduces biometric data risk. [R18] Actual facial identity verification should remain a separate, higher-risk research branch.

Progressive verification

Role	Minimum research verification
Buyer	Eduvos identity/email/student status verification
Seller	Buyer verification + government ID + seller onboarding
Courier	Reusable ID verification + transport mode + courier onboarding + payout setup
Transfer Courier	Courier requirements + car/motorbike + transfer opt-in + route eligibility
Hub Operator	Staff identity + role assignment + staff-level MFA/passkey
Admin/Finance	Staff identity + high-assurance role + step-up authentication + audit logging

The backend must enforce RBAC and permission checks. The client may hide buttons, but the server is the source of truth for who can scan packages, release payouts, approve refunds, view Incubation indicators or configure fee policies.

18. Operations Platform role model

The Operations Platform is one application with role-based modules, not separate staff apps. This reduces maintenance overhead and keeps staff workflows consistent. However, the backend must ensure users only see and act on authorised resources. A Hub Operator should not automatically see financial operations or Incubation health indicators.

Operations modules

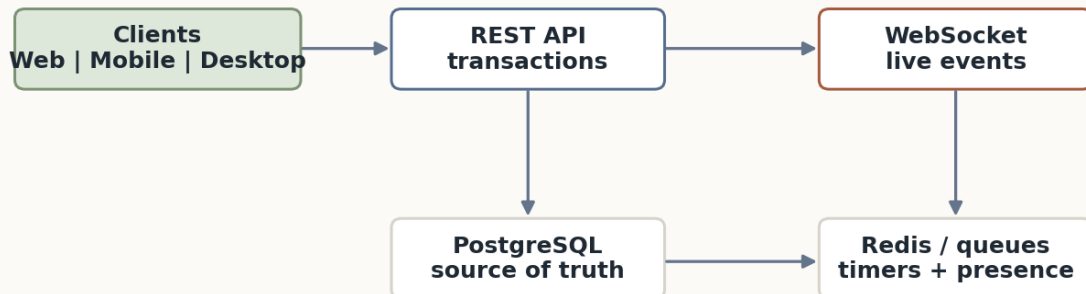
Module	Main workflows
Hub Operations	Package intake, condition check, rejection, return handling, pickup release, QR scanning
Transfer Operations	Transfer windows, route capacity, manifest locking, departure/arrival reconciliation
Incubation Hub	Demand intelligence, opportunity signals, business health indicators, interventions, 30-day plans
Moderation and Disputes	Reports, prohibited items, evidence review, appeals, buyer/seller enforcement
Finance Operations	Refund exceptions, payout status, ledger reconciliation, suspicious withdrawal reviews
Admin	Campuses, hubs, categories, capacity policies, fee rules, roles, system configuration

Operations actions require stronger audit logging. Every staff action should record actor, role, resource, old value, new value, timestamp, device/session and reason where needed.

19. Real-time system design

EduHackers requires a real-time layer because seller offers, courier offers, live tracking, chat, hub scans, transfer progress and service slot timers cannot depend only on page refreshes. However, WebSockets should not become the source of truth. The database and REST API remain authoritative; WebSockets distribute live state changes to connected clients.

REST + WebSocket split: authoritative state versus live updates



If live connection drops: reconnect -> fetch authoritative current state via REST -> resume live stream.

Figure 6. REST handles authoritative mutations and recovery reads; WebSockets deliver live events while the app is active.

Core live events

Event group	Examples
Dispatch	seller.offer.created, seller.offer.expired, courier.offer.created, request.locked
Courier	courier.assigned, courier.location.updated, eta.updated, courier.arrived
Hub	package.hub_received, package.rejected, package.condition_exception
Transfer	transfer.manifest_locked, transfer.departed, transfer.arrived, transfer.reconciled
Service booking	slot.held, slot.released, seller.declined, rematch.offered
Payments	payment.authorised, refund.started, payout.processing, wallet.balance_changed
Operations	dispute.opened, report.created, intervention.approved

The POF should test WebSocket disconnects. If the connection drops, the UI should show reconnecting, reconnect, then call REST to fetch the current authoritative state before resuming live updates.

20. Database and data model research

PostgreSQL with PostGIS is the recommended baseline because the system is strongly relational, transaction-heavy and location-aware. PostGIS ST_DWithin supports distance checks using spatial indexes, which is directly relevant for hub, courier and campus-radius matching. [R12] PostgreSQL row-level security can add another layer of data access control, although application-level RBAC remains mandatory. [R15]

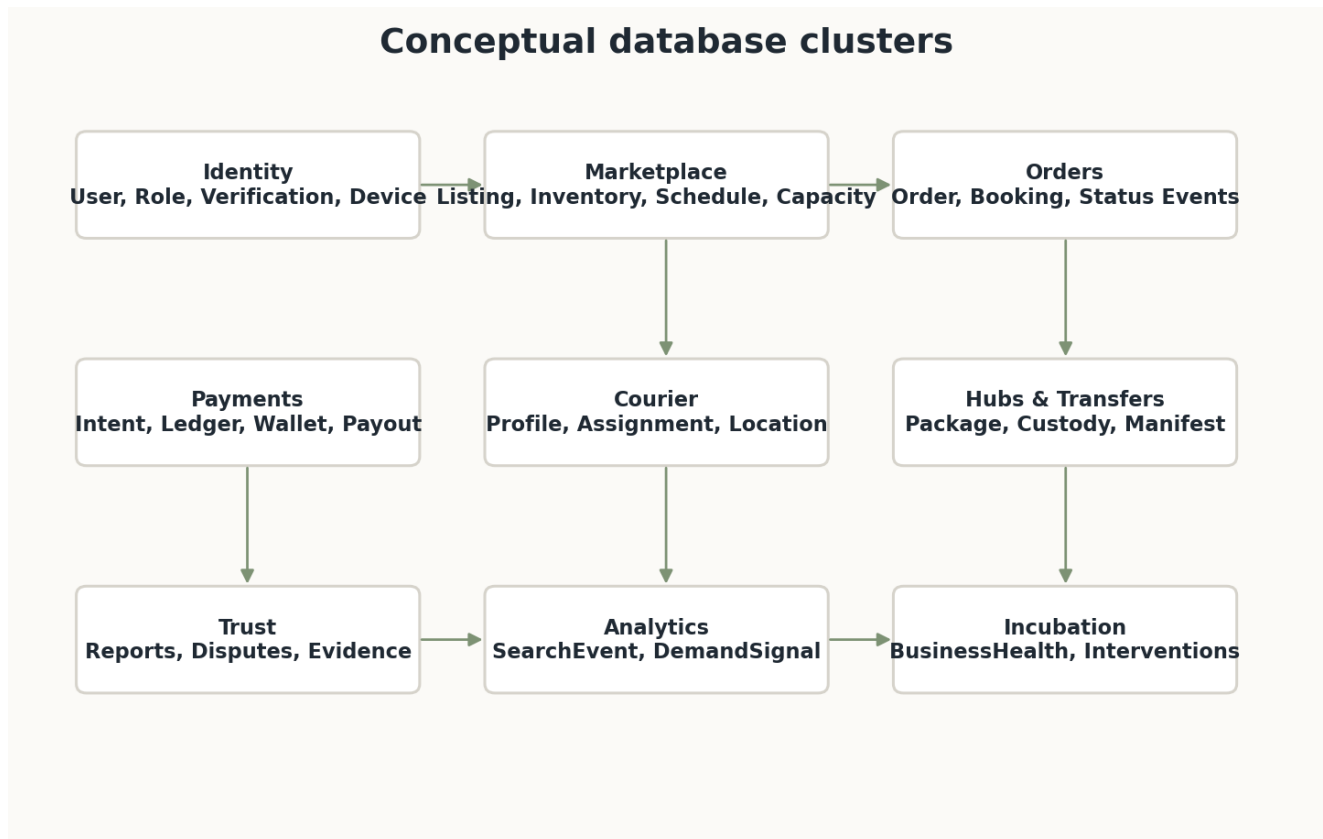


Figure 7. Conceptual clusters. The exact schema should evolve from POF requirements and migration files.

High-value data rules

Rule	Reason
Use UUIDs or collision-resistant IDs for public identifiers	Avoid exposing sequential business volume
Store money in integer minor units, not floating point	Prevents precision errors
Use transaction/locking for accept flows	Prevents double acceptance, overselling and double booking
Snapshot policy versions	Enables audit of old fees, capacity, pricing and transfer rules
Separate internal identity from public profile	Protects ID/student number and verification data
Use append-only events for custody and finance where possible	Improves dispute reconstruction

21. API research map

The API should be designed around domain contracts rather than one generic order endpoint. Product, service, courier and transfer flows have different state machines. A shared Order model can exist internally, but external API contracts should remain explicit so clients do not guess which state transitions are allowed.

Representative API groups

API group	Example endpoints / events
Identity	POST /auth/login, POST /roles/seller/activate, POST /roles/courier/activate
Listings	POST /listings, PATCH /inventory, PATCH /seller-category-availability
Requests	POST /product-requests, POST /requests/{id}/accept, POST /requests/{id}/decline
Services	POST /service-requests, POST /slots/hold, POST /bookings/{id}/confirm
Courier	POST /courier/offers/{id}/accept, PATCH /delivery/{id}/location, POST /delivery/{id}/pin
Hubs	POST /packages/{id}/scan, POST /packages/{id}/condition, POST /packages/{id}/reject
Transfers	POST /transfer-batches/{id}/lock, POST /transfers/{id}/depart, POST /transfers/{id}/reconcile
Payments	POST /payments/authorise, POST /refunds, POST /withdrawals, GET /ledger
Operations	GET /ops/hubs/{id}/queue, POST /disputes/{id}/decision, PATCH /fee-rules

Every state-changing endpoint should validate current state, role permissions, object ownership, and any timing/capacity constraints server-side. The client should never directly set final states like `MATCHED_LOCKED` or `PAYOUT_RELEASED` without backend validation.

22. Technology stack research baseline

The recommended research stack is TypeScript end-to-end, with Next.js for web, React Native/Expo for mobile, Tauri for Windows/macOS desktop, NestJS for backend, PostgreSQL/PostGIS for data, Redis/BullMQ for live timers/queues and GitHub Actions for CI/CD. This is not chosen because it is trendy. It is chosen because it keeps one language across the team, supports live marketplace flows and provides strong ecosystem coverage for web, mobile, desktop and backend.

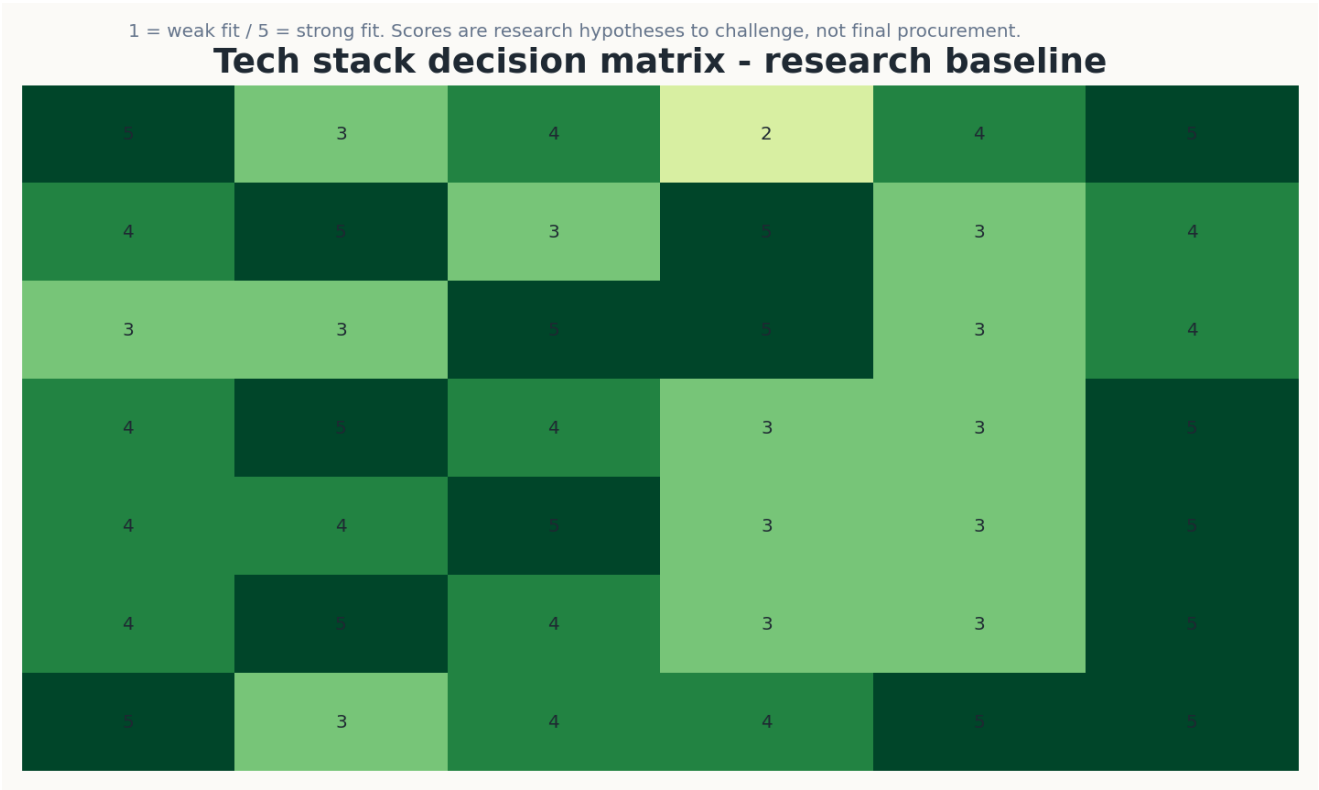


Figure 8. Decision matrix scores are research hypotheses. The team should challenge them against actual skills and build constraints.

Stack decisions still open

Area	Open decision
Backend hosting	Vercel functions, Railway, Render, Fly.io, AWS, Supabase Edge or other platform
Realtime gateway	NestJS WebSocket server, managed realtime, or separate gateway process
Object storage	Supabase Storage, S3-compatible bucket, Cloudflare R2 or provider storage
Payments	Paystack/Ozow/Payfast/other provider research; no final provider locked
Maps/location	Provider decision depends on cost, mobile SDK and privacy controls
Auth	Eduvos SSO/API availability must be confirmed; identity abstraction should hide vendor choice

23. GitHub engineering workflow

GitHub should be the engineering source of truth, but not the place for all project assets. Code, migrations, API contracts, tests, architecture decisions and technical documentation belong in GitHub. Figma remains the design source of truth. Miro remains the research/workshop system. Large video, PDF, image or dataset assets should live in object storage or drive, because GitHub enforces large object limits and recommends Git LFS for large files. [R16]

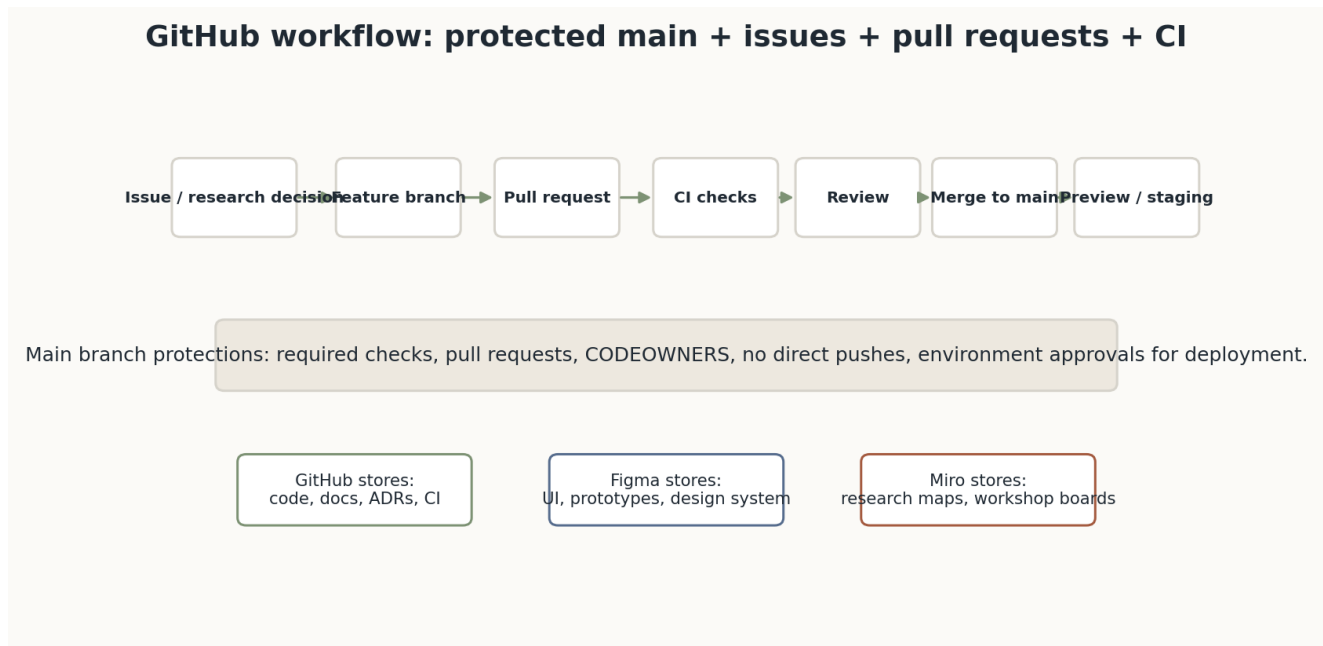


Figure 9. Protected main, feature branches, pull requests, CI and review create a sane engineering workflow for a six-person team.

Recommended repository structure

Use an EduHackers GitHub Organization and a private monorepo to start. The monorepo makes shared API contracts, event types, validation schemas and design tokens easier to keep consistent across Marketplace, Operations, mobile, desktop and backend. If the system later grows, logical service boundaries can be split into separate repositories.

Path	Purpose
apps/marketplace-web, marketplace-mobile, marketplace-desktop	Student-facing client applications
apps/operations-web, operations-mobile, operations-desktop	Role-based staff applications
services/gateway, identity, commerce, dispatch, logistics, payments	Backend service modules / deployable units
packages/api-contracts, event-types, validation, permissions	Shared contracts used by all clients and services
packages/database	Schema, migrations, seed data, query helpers
docs/architecture, docs/api, docs/security, docs/decisions	Engineering documentation and architecture decision records
.github/workflows	CI, tests, lint, typecheck, build, deployment automation

24. Branching, pull requests and team rules

The team should use a protected main branch with short-lived feature branches. Every meaningful change goes through a Pull Request, even if one developer is doing most of the work. GitHub Actions should run linting, type checks, unit tests, schema validation and build checks before merge. GitHub supports required status checks and branch protection rules, which can prevent broken changes from entering protected branches. [R17]

Basic branch pattern

Branch or artifact	Rule
main	Always releasable; protected; no direct push
feature/*	One focused feature or research spike
fix/*	Bug fix branch
docs/*	Architecture/docs changes
release/*	Optional later when deployment process matures
Pull Request	Must include summary, screenshots/recording for UI, linked issue, tests run and risk notes
CODEOWNERS	Routes review to Developer/Research Lead and relevant teammate based on changed area

Do not treat GitHub as only a backup. Issues should represent research questions, POF tasks, bugs, API contract decisions and UX handoff items. Pull Requests should document what changed and why.

25. Developer and UX/UI interface contract

This project can fail if the developer and UX/UI workstreams operate separately. The developer defines states, API responses, permission boundaries and timing rules. UX defines how users understand, trust and act on those states. The bridge between them should be an explicit interface contract.

Developer-informed wireframe research: states before styling

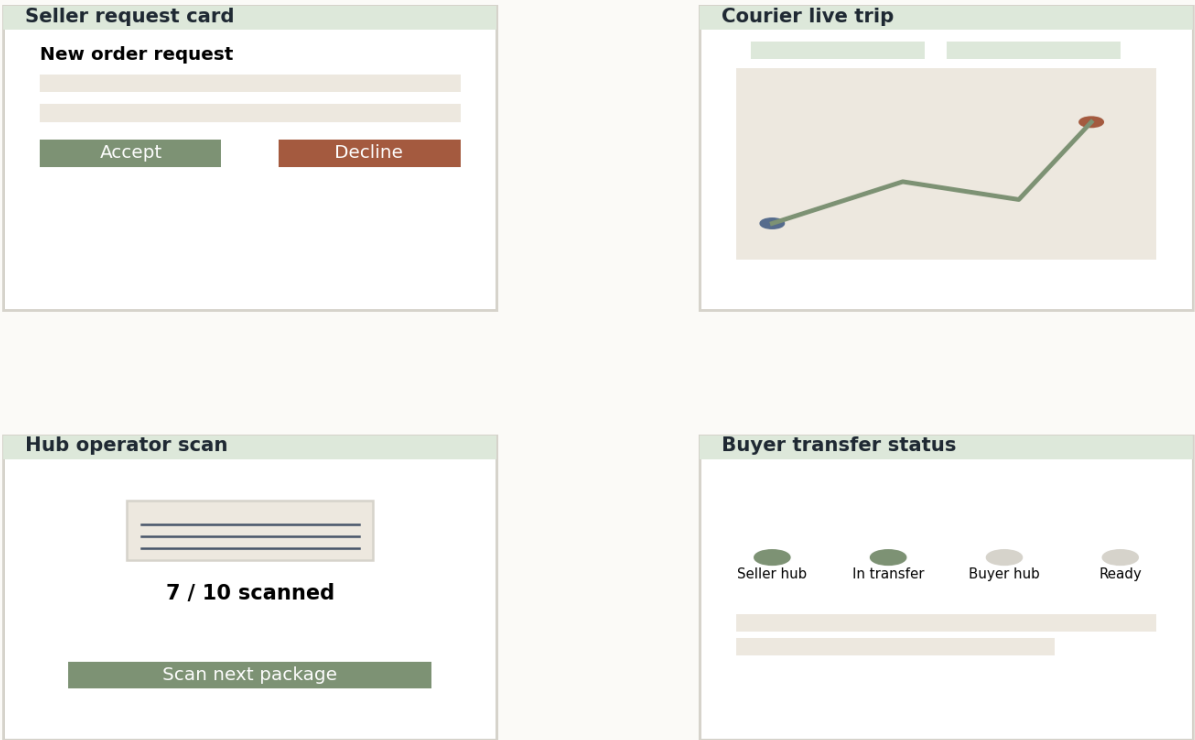


Figure 10. These are not final UI designs. They are developer-informed wireframes showing required states before styling.

Interface contract format

Backend state / event	Required UI behaviour
seller.offer.created	Show incoming request with item, quantity, method, value and countdown
request.locked	Remove other seller offers; show accepted confirmation
package.hub_received	Buyer sees package awaiting courier; courier matching begins
courier.assigned	Buyer/seller see courier details and ETA; message/call becomes available
delivery.at_dropoff	Buyer sees arrival alert and PIN reminder
slot.temporarily_held	Buyer sees proposed service slot and expiry timer
seller.at_capacity	Listing stays visible with Busy/Currently at capacity and alternatives
transfer.manifest_locked	Transfer courier and Hub Operator see locked package count
wallet.withdrawal.processing	User sees amount pending and cannot spend same balance twice

26. Notification and communication architecture

Core notifications should use push, in-app notifications and Eduvos/university email. SMS should not be part of the first research baseline unless validation later proves it is necessary. Critical operational notifications should always remain enabled: active requests, active delivery, payment/refund/payout changes, payout-destination security events, hub handoff, transfer changes and account restrictions.

Expo provides push notification capabilities for iOS and Android, including tokens, foreground/background handling and notification interaction support. [R9] The research stack should use push for background events, WebSockets for active live updates, in-app notifications as the activity centre and university email for important fallback records.

Communication permissions

Communication pair	When allowed	Expiry
Buyer - Seller	Active order/service/request context	Ends after configured post-completion period
Buyer - Courier	Active delivery only	Ends after delivery/failed-delivery resolution
Seller - Courier	Food/direct pickup and relevant delivery contexts	Ends when pickup is complete
Hub Operator - Courier	Transfer or pickup exception context	Ends after reconciliation
Operations - User	Dispute, compliance, payout or support case	Ends when case closes

Do not expose personal phone numbers by default. Use in-app messaging and in-app/masked calling where feasible. Messages should be tied to a transaction context rather than becoming permanent personal chat.

27. Security, privacy and POPIA-aligned design

The platform will process identity, government-ID verification status, location events, messages, order history, financial ledger records, payout destinations, package condition evidence and disputes. That makes privacy and security core architecture concerns, not final-stage legal copy. POPIA requires responsible processing of personal information, and the platform should minimise what it collects, stores and exposes by role.

Security controls

Control	Research requirement
Server-side RBAC/ABAC	Every sensitive endpoint checks role, permission, ownership and resource scope
Step-up authentication	Required for payout destination changes, high-risk staff actions, large withdrawals if flagged
Audit logging	Staff and sensitive financial/custody actions must be auditable
Object storage controls	Dispute/package evidence must use secure URLs and access checks
Location minimisation	No continuous courier tracking outside accepted active tasks
Passkeys/device biometrics	Research preferred login/reauth path instead of custom face database
Database row-level security	Consider as defence-in-depth; do not replace app-level permissions
Input validation and state guards	Use shared schemas and state-transition validation

Privacy design should keep public seller profiles separate from verified identity records. Staff views should show only what is necessary for their role. Transfer couriers should see package metadata, not buyer or seller private details.

28. Observability, audit and operational monitoring

A live marketplace cannot be debugged from user complaints alone. The POF and future build need structured logs, metrics, traces and audit events. OpenTelemetry provides a vendor-neutral standard for collecting traces, metrics and logs across systems, making it a suitable research direction for observability. [R14]

Operational metrics

Metric	Why it matters
seller_match_time	Measures whether dispatch is fast enough
batch_timeout_rate	Shows whether batch sizes/timers are wrong
simultaneous_accept_conflicts	Tests race condition protections
courier_time_to_hub	Shows pickup SLA performance
courier_wait_at_seller	Food preparation accuracy indicator
hub_rejection_rate	Packaging/compliance problem indicator
manifest_exception_rate	Transfer trust and custody metric
payment_authorisation_failures	Payment UX/provider fit indicator
wallet_withdrawal_failures	Payout operations and provider reliability indicator
realtime_disconnect_recovery	Measures live system robustness

Audit logs and telemetry are not the same thing. Telemetry helps engineering see health and performance. Audit logs preserve who did what to regulated, financial, operational or identity-sensitive records.

29. Proof of Flow / Proof of Functionality

The team should build a POF before trying to implement the full app. The POF should demonstrate the hardest system flows with realistic state transitions, not polished feature breadth. The objective is to prove that EduHackers can handle dispatch, payment states, package custody and operational visibility correctly.

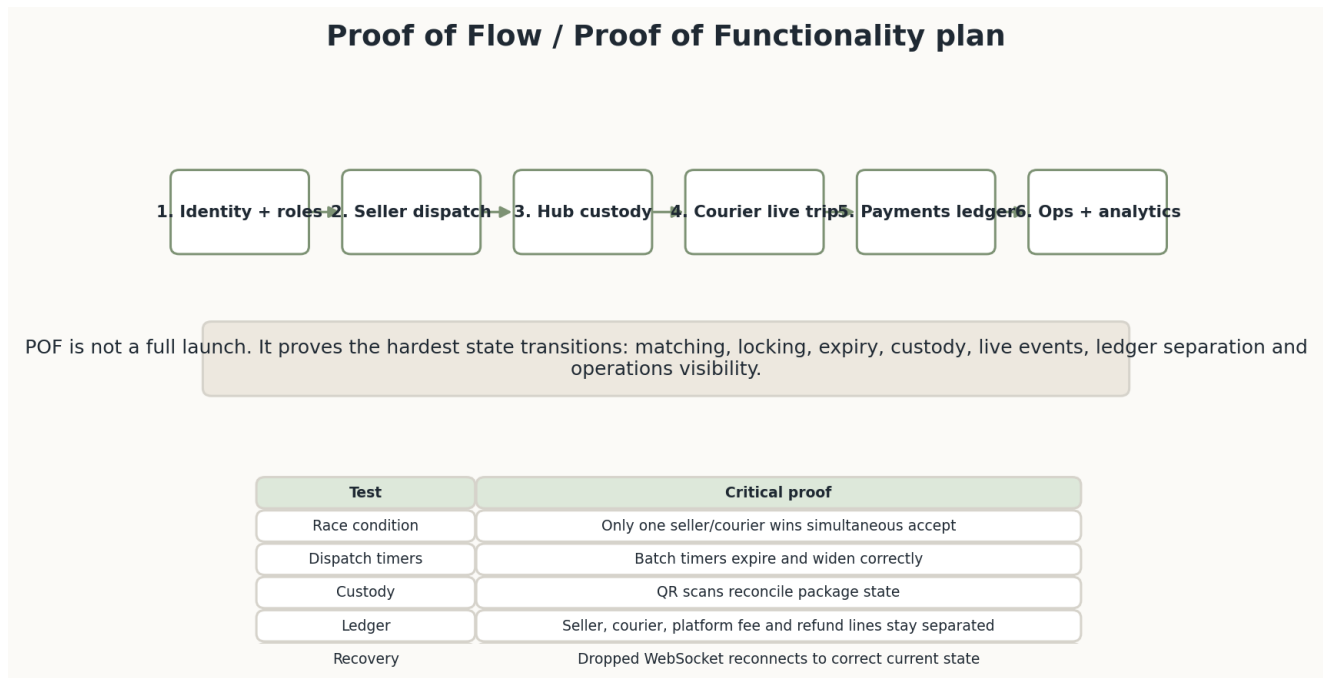


Figure 11. POF scope should test the hard backend behaviours first, then UX polish and broader product modes later.

POF minimum path

POF path	Must prove
Buyer requests product	Listing price snapshot, seller eligibility and stock checks
Seller batch dispatch	Staggered offers, expiry, decline, atomic accept lock
Product payment window	Payment pending, expiry and rematch on non-payment
Hub intake	QR scan, condition check, accept/reject, 24-hour correction flow
Courier delivery	Courier dispatch after hub received, live tracking, chat/call mock, QR + PIN handoff
Failed delivery	10-15 minute wait, hub return, 48-hour pickup hold
Operations view	Hub queue, package state, scan history, audit trail
Ledger view	Seller amount, courier amount, platform fee and refund state separated

The POF can use mocked payment and identity providers while preserving the same state transitions expected from real providers. That gives the team a credible demo without prematurely taking real money or processing real identity documents.

30. Roadmap and build phases

The development roadmap should align to research risk, not only visual completeness. The system should first prove state correctness and the core loop, then broaden features. The earlier marketplace research already established the bigger product universe: products, services, food, rentals, events, reels, group buying, Incubation intelligence and more. The developer roadmap below narrows what must be proven first.

Research-to-build roadmap

Phase	Developer focus	Output
Week 1: Architecture validation	Confirm stack, repository, API contracts, data model and POF scope	Technical decision log + repo skeleton
Week 2: Identity and roles	Eduvos identity assumption, buyer/seller/courier/staff roles, RBAC permissions	Auth flow mock + permissions tests
Week 3: Dispatch engine	Seller batch matching, timers, atomic accept, capacity rules	Dispatch POF with race-condition tests
Week 4: Orders and hubs	Product order state machine, hub scan, package condition, rejection flow	Hub operations POF
Week 5: Courier and realtime	Courier offers, live trip, QR/PIN, delivery states, notification events	Courier POF with WebSocket reconnect
Week 6: Payments ledger mock	Authorisation states, fees, seller/courier ledger, refund states	Auditable ledger demo
Week 7: Operations and analytics	Staff modules, audit logs, demand signals, Incubation view mock	Operations dashboard
Week 8: Integrated demo	End-to-end scenario from request to delivery to analytics	Team pitch demo + technical risk register

31. What should be researched next with the team

The next team session should not reopen every product idea. It should validate whether this technical approach supports the product vision and whether UX, product, business and pitch can work with these system assumptions. The developer can present the diagrams, explain the state machines and ask each role for targeted feedback.

Questions for UX/UI

Area	Question
Dispatch	How urgent should request cards feel without causing stress?
Capacity	How do we show Busy, At Capacity and Notify Me without making sellers look unreliable?
Hub flows	How does seller drop-off and hub scan stay quick for normal cases?
Courier	Where do message/call, ETA and PIN appear without clutter?
Transfer	How much of hub-to-hub movement should the buyer see?
Wallet	How do we explain available, processing, paid and withdrawable without looking like a bank app?
Operations	How does one Operations app adapt to very different staff roles?
Accessibility	How do countdowns, urgent alerts and live maps work for low vision and screen readers?

Questions for Product / Business / Pitch

Role	Questions
Product Lead	Which POF path best proves the marketplace loop? What can be cut without weakening the demo?
Business Analyst	Which fees, payout rules and operational costs need validation before claiming sustainability?
Marketing Lead	How do we explain "Uber-like" without implying we are copying Uber?
Project Manager	How do we sequence developer, UX and research milestones so no stream blocks another?
Pitch Lead	Which technical proof points matter most to judges: dispatch, trust, payment, Incubation intelligence or all together?

32. Differentiation summary

Existing platforms already demonstrate many individual pieces: dynamic dispatch, food-delivery live tracking, merchant busy mode, order pauses, marketplace splits, GitHub CI/CD, passkeys, spatial databases and real-time communication. EduHackers should not pitch these individual components as if they are new.

The stronger differentiation is the system combination: verified Eduvos identities, student sellers, campus hubs, QR/PIN custody, category-aware fulfilment, hub-to-hub transfer windows, student courier earnings, staff operations, protected ledger states, demand intelligence and Incubation Hub intervention. This is a campus entrepreneurship operating system, not just an Uber clone or a listing app.

The technical research therefore recommends building a POF that proves the closed operational loop: demand is created, a seller is matched, the package is protected through custody events, a courier fulfils the delivery, payments are traceable, Operations can see exceptions, and system data can feed future marketplace and Incubation decisions.

33. Open risks and assumptions

Risk / assumption	Why it matters	Research response
Eduvos SSO/API availability	Identity architecture depends on institutional integration	Design identity abstraction; validate with Eduvos IT
Payment provider capabilities	Authorisations, splits, incremental charges and withdrawals vary	Keep provider adapter; build mock provider for POF
Hub staffing and opening hours	Hubs are central to product logistics	Validate actual campus operations; make hub schedules configurable
Student courier legality/insurance	Courier model may trigger legal and operational requirements	Research with Eduvos and provider/legal input
Scope overload	Platform is broad and can exceed Hack Jam capacity	Build POF path, not every feature
Dynamic pricing ethics	Opaque pricing can harm trust	Use bounded, transparent modifiers and audit snapshots
Operations workload	Staff scanning/condition checks may be too slow	Test hub workflows and count scan time
Privacy exposure	Closed campus makes identity/location sensitive	Minimise data by role; use opaque package tokens

34. Reference list

- R1.** Eduvos Hack Jam 2026 Brief. Uploaded source of truth for challenge, roles and judging criteria.
- R2.** Next.js documentation. <https://nextjs.org/docs>
- R3.** NestJS microservices documentation. <https://docs.nestjs.com/microservices/basics>
- R4.** Uber Marketplace: Surge pricing. <https://www.uber.com/us/en/marketplace/pricing/surge-pricing/>
- R5.** DoorDash Merchant Help: Store status, Busy and Pause. <https://help.doordash.com/en-us/merchants/article/what-is-my-store-status>
- R6.** Uber Eats Merchant Academy: Accepting orders and Busy Mode. <https://merchants.ubereats.com/au/en/academy/orders/>
- R7.** Uber Eats blog: Delivery person wait time and ready-for-pickup tools. <https://www.uber.com/au/en/blog/delivery-person-wait-time/>
- R8.** Tauri capabilities documentation. <https://tauri.app/security/capabilities/>
- R9.** Expo Notifications documentation. <https://docs.expo.dev/versions/latest/sdk/notifications/>
- R10.** BullMQ documentation: delayed jobs and queues. <https://docs.bullmq.io/guide/jobs/delayed>
- R11.** Redis Streams documentation. <https://redis.io/docs/latest/develop/use-cases/streaming/>
- R12.** PostGIS ST_DWithin documentation. https://postgis.net/docs/ST_DWithin.html
- R13.** Paystack Transaction Split API. <https://paystack.com/docs/api/split/>
- R14.** OpenTelemetry documentation. <https://opentelemetry.io/docs/>
- R15.** PostgreSQL Row Security Policies. <https://www.postgresql.org/docs/17/ddl-rowsecurity.html>
- R16.** GitHub repository limits. <https://docs.github.com/en/repositories/creating-and-managing-repositories/repository-limits>
- R17.** GitHub protected branches. <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>
- R18.** FIDO Alliance passkeys. <https://fidoalliance.org/passkeys/>
- R19.** Stripe Connect pricing and marketplace platform fees. <https://stripe.com/en-ca/connect/pricing>
- R20.** Protection of Personal Information Act (POPIA). <https://www.gov.za/documents/protection-personal-information-act>

35. Appendices

Appendix A: Suggested architecture decision records

ADR	Decision to research / record
ADR-0001	Two application families: Marketplace and Operations Platform
ADR-0002	Hybrid service-oriented/event-driven architecture
ADR-0003	PostgreSQL/PostGIS as primary database
ADR-0004	Seller dispatch: staggered batches and atomic locks
ADR-0005	Campus hub custody model for physical products
ADR-0006	Category-aware fulfilment policies
ADR-0007	Payment provider abstraction and ledger model
ADR-0008	GitHub monorepo and protected-main workflow
ADR-0009	Realtime event contract and reconnect behaviour
ADR-0010	Identity assurance levels and progressive verification

Appendix B: Initial event dictionary

Event	Purpose
request.created	Buyer created product/service demand
seller.offer.created	Seller dispatch offer started
request.locked	One seller won atomic accept
payment.authorised	Funds authorised or protected
package.hub_received	Hub accepted package custody
package.rejected	Hub refused package intake
courier.assigned	Courier won dispatch
delivery.pin_verified	Buyer handoff complete
transfer.manifest_locked	Batch closed for departure
transfer.reconciled	Destination hub verified manifest
wallet.withdrawal_requested	User requested payout
seller.capacity_changed	Capacity changed through policy/busy/recovery
demand.signal_created	Unmet demand became opportunity signal

Appendix C: Initial test cases

Test case	Expected result
Three sellers accept simultaneously	Only one seller locks order; two receive already accepted response
Buyer payment expires after seller accept	Stock releases and request returns to matching pool
Courier accepts then no movement	Warning, then rematch after threshold
Hub rejects package	24-hour correction flow starts and buyer notified
Destination hub missing package	Transfer payout held, investigation opens
WebSocket disconnect during delivery	Client reconnects and fetches current state via REST
Service slot held by two buyers	Only one can confirm; other sees released/changed slot
Payout destination changed	Step-up verification required and audit event created

Test case	Expected result
Seller reaches capacity	Listings remain visible, matching exclusion begins, Notify Me queue available

36. Architecture options compared

The research should compare architecture patterns instead of blindly choosing one. A modular monolith gives faster delivery and simpler transactions. Microservices give isolation and independent scaling but introduce distributed transactions, deployment overhead and harder debugging. Serverless gives elasticity but can complicate long-lived WebSockets, queue consumers and stateful dispatch. A hybrid service-oriented architecture offers a realistic compromise for an eight-week build.

Option	Advantages	Risks	Research verdict
Modular monolith	Fastest to build, simple transactions, easy local development	Can become tangled if module boundaries are weak	Good for initial POF if modules are enforced
Microservices	Independent deploy/scale, clear service ownership	Distributed transactions, observability burden, DevOps cost	Research but avoid overbuilding in Hack Jam
Serverless	Low ops, scales per request	Cold starts, WebSocket complexity, timers/state orchestration issues	Use selectively, not as whole-platform assumption
Hybrid service-oriented	Clear domains, separable services where needed, manageable complexity	Requires discipline in contracts and event design	Current baseline research direction

The decisive criterion is not architectural fashion. It is whether the chosen structure can prove dispatch, custody, payments and operations without blocking the team in infrastructure work.

37. Deployment environments and release flow

The platform should have at least local, preview, staging and production environments. Local is for individual development. Preview is created from pull requests where practical. Staging should mirror production configuration as closely as possible without real money or real identity documents. Production should only exist after legal, payment and institutional risks have been addressed.

Environment	Purpose	Data rule	Who uses it
Local	Developer build and tests	Seeded fake data	Developer
Preview	Pull-request validation	Ephemeral fake data	Team reviewers
Staging	Integrated demo and UAT	Representative fake/sandbox data	Whole team and possibly Eduvos reviewers
Production	Live platform	Real data only after approvals	Real users

Payment providers, push notifications, email and identity verification must all support sandbox mode. The POF should never process real funds or documents even if the screens look production-ready.

38. Idempotency, retries and duplicate prevention

Idempotency is critical because mobile networks are unreliable and users can tap the same action twice. Accepting a seller offer, charging a payment, scanning a package or confirming a delivery should not create duplicate side effects if the request is retried.

Action	Idempotency rule
Seller accepts request	Idempotency key per seller offer; backend lock determines winner
Courier accepts delivery	Only one active assignment per delivery task
Payment authorisation	Provider reference and idempotency key prevent duplicate charges
Package scan	Repeated scan updates existing custody event or returns already scanned state
PIN verification	PIN can be used once; repeated attempt returns already completed
Withdrawal request	Balance reserve and payout reference prevent duplicate payout

The POF should intentionally simulate double taps, connection loss and repeated API calls. If those tests fail, the platform is not ready for real users.

39. Database indexes and transaction strategy

The database design should be guided by critical queries. The matching engine needs fast lookup by campus, category, availability, capacity and location. The operations platform needs fast hub queues, package custody histories and transfer manifests. Payments need exact ledgers and immutable audit trails.

Area	Index or constraint idea
Listings	category_id, campus_id, active, stock_available
Seller capacity	seller_id + category_id uniqueness; active order count derived or maintained transactionally
Service slots	seller_id + start_at + status; unique booked slot constraint
Courier matching	transport_mode, active, current campus/zone, reliability tier
Geographic matching	PostGIS spatial index for courier/hub/campus distance
Package custody	package_id + timestamp; transfer_batch_id + package_id uniqueness
Ledger	user_id + created_at; transaction_id; provider_reference uniqueness
Audit	resource_type + resource_id + created_at

Race-sensitive flows should use database transactions or equivalent locks. Examples include seller accept, courier accept, service slot hold, wallet withdrawal reserve and transfer manifest lock.

40. Timers, queues and scheduled jobs

Many EduHackers behaviours are timer-driven. Seller offers expire, service slots release, payment windows close, correction grace periods expire, hub pickup holds expire, courier inactivity warnings trigger and transfer windows close. These should be driven by a queue/scheduler system rather than fragile client timers.

Timer / scheduled job	Purpose
seller_offer_expiry	Move to next seller batch or campus expansion
buyer_payment_window_expiry	Release stock/slot and reopen matching
service_slot_hold_expiry	Return slot to availability
courier_inactivity_warning	Warn courier if not moving toward hub
hub_correction_deadline	Cancel/refund if seller does not fix rejected package
pickup_point_48h_expiry	Move package to uncollected/return-to-seller flow
transfer_window_cutoff	Lock manifest and stop adding packages
daily_payout_batch	Process lower-cost scheduled payouts

Client timers are only visual. The backend timer is authoritative. When a user opens the app, the client should fetch the current server state and render the correct countdown from server timestamps.

41. Integration adapter strategy

The system must assume that vendors can change. Payments, identity verification, email, push, maps, storage and analytics should be behind adapters. This prevents the team from hard-coding a payment provider or map provider into core domain logic.

Integration	Adapter responsibilities
Payment provider	Authorise, capture, refund, split, payout, webhook verification
Identity verification	Start check, receive result, store verification status
Eduvos identity/SSO	Verify active student/staff status, campus and email
Email provider	Send university email notifications and templates
Push provider	Send device push, handle tokens and failure states
Map/location provider	Route estimates, geocoding, distance/time estimates
Object storage	Signed uploads/downloads, evidence access control, retention
Analytics	Export event summaries without leaking unnecessary personal data

Every provider webhook should be verified cryptographically where supported. Webhook events should be stored and processed idempotently, because providers may retry events.

42. GitHub project board and issue taxonomy

A private EduHackers GitHub organization should use one project board for build work, with labels that map to both product and technical domains. This lets the research lead track architecture, frontend, backend, operations and QA without losing decisions in chat messages.

Label	Meaning
domain:identity	Auth, verification, roles, passkeys
domain:dispatch	Seller/courier matching and timers
domain:logistics	Hubs, packages, transfer batches and custody
domain:payments	Payments, wallet, ledger, fees and payouts
domain:ops	Operations Platform modules
type:research	Question or spike, not implementation
type:feature	Buildable behaviour
type:bug	Defect in implementation
type:decision	Architecture or product decision needed
risk:high	Requires early proof or external confirmation

Every GitHub issue should contain context, expected outcome, acceptance criteria, linked research source and owner. For POF tasks, the acceptance criteria should be behavioural, not vague.

43. Testing strategy

Testing should focus on state transitions, permissions and race conditions first. Visual snapshot tests and UI testing matter, but the project is riskier if the backend allows double acceptance, missing package reconciliation or incorrect payouts.

Test type	What it should cover
Unit tests	Pricing rules, capacity rules, matching scoring, state guards
Integration tests	Database transactions, API endpoints, auth permissions
Race tests	Simultaneous seller/courier accepts, simultaneous withdrawals, slot double booking
Contract tests	API payloads and event schemas shared with clients
End-to-end POF tests	Buyer request to seller accept to hub scan to courier delivery
Security tests	RBAC bypass attempts, object ownership, evidence URL access
Offline/reconnect tests	Mobile app reconnect and authoritative state refresh
Operations tests	Hub scan mismatches, transfer reconciliation, package rejection

The first demo should include at least one deliberate exception flow. A perfect happy path is less convincing than showing that the system handles a rejected package, late courier or transfer discrepancy correctly.

44. Accessibility and performance constraints

The platform will use countdowns, live maps, urgent requests and operational dashboards. Accessibility cannot be left to final styling. Low-vision users, keyboard users and screen-reader users need readable status, not color-only signals. Performance also matters because students may use ordinary mobile data and older phones.

Constraint	Research response
Urgent notifications	Provide text, vibration/sound where permitted, and screen-reader-friendly labels
Color-coded status	Never rely on color alone; include text status and icons/labels
Countdowns	Server-based times with accessible announcements at key thresholds
Maps	Offer text status and ETA fallback when map interaction is not usable
Large dashboards	Use filtering, role-based home views and exportable detail
Low bandwidth	Core order/dispatch screens must work without heavy media
Motion graphics	Use restraint and respect reduced-motion settings

Accessibility should be measured with keyboard navigation, screen reader checks and contrast testing during the design/build cycle, not after the pitch deck is finished.

45. Demo story for the team

The developer research should be presented as a system story, not only a stack list. The story below is a suggested team walkthrough that connects product value to backend architecture.

Demo chapter	What to show
1. Buyer demand	Buyer requests a listed product at fixed price
2. Seller dispatch	Staggered seller batch, first accept wins, other offers close
3. Payment window	Buyer pays after seller accepts; price snapshot preserved
4. Hub intake	Seller drops at hub, Hub Operator scans QR and records condition
5. Courier delivery	Courier dispatched after hub receipt, live location and chat/call enabled
6. PIN handoff	Buyer provides one-time PIN and delivery completes
7. Operations view	Staff see custody timeline and status history
8. Analytics	Demand and fulfilment events become marketplace insight

This demo story proves why architecture matters. Every screen has a backend state behind it. Every trust claim has an event, audit record or ledger entry behind it.

46. Developer backlog by epic

Epic	Initial backlog items
Identity and roles	Auth mock, role claims, permission guards, student/staff verification placeholder
Marketplace listings	Create listing, price snapshot, inventory, category schedule, capacity
Seller dispatch	Batch creation, stagger timers, accept/decline, atomic lock, expiry
Orders	Product order state machine, service booking state machine, status history
Hub operations	QR package, hub intake, condition check, rejection, 24-hour correction
Courier	Courier onboarding, transport mode, offers, live trip states, PIN handoff
Transfers	Transfer windows, manifest, package scans, reconciliation
Payments ledger	Mock authorisation, fee snapshot, ledger entries, refund and payout states
Realtime	WebSocket gateway, event schemas, reconnect recovery
Operations app	Hub queue, transfer queue, audit timeline, exception case view

Each backlog item should have acceptance criteria. For example, "seller dispatch works" is not acceptable. "Two simultaneous accepts result in exactly one locked request" is acceptable.

47. Final developer research position

EduHackers can research and build an Uber-inspired marketplace dispatch system, but the opportunity is not to copy Uber. The opportunity is to build a campus-specific commerce and operations system where dispatch, custody, student delivery, protected payments and Incubation intelligence work together.

The current best direction is a hybrid service-oriented platform with a student Marketplace app family and a role-based Operations Platform. PostgreSQL/PostGIS, TypeScript, Next.js, React Native/Expo, Tauri, NestJS, Redis/BullMQ and GitHub Actions are a sensible research baseline, but every provider decision remains open until the team validates cost, team skill, Eduvos constraints and POF feasibility.

The next immediate action is to convert this research into a team walkthrough, then define the POF backlog in GitHub. The POF should prove the state machines and trust mechanisms first. After that, UI polish, broader marketplace modes and advanced features become much safer to build.